Тестирование программного обеспечения

ТЕСТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

- ✓ Поиск ошибок
- ✓ Проверка соответствия ПО требованиям и здравому смыслу
- ✓ Оценка работоспособности ПО
- ✓ Способ контролировать качество ПО

Тестирование программного обеспечения — процесс анализа программного средства и сопутствующей документации с целью выявления дефектов и повышения качества продукта.

В глоссарии **ISTQB*** нет термина «тестирование ПО», который широко используется в русском языке. Там есть лишь термин «тестирование (**testing**)».

Тестирование (testing) — процесс, содержащий в себе все активности жизненного цикла, как динамические, так и статические, касающиеся планирования, подготовки и оценки программного продукта и связанных с этим результатов работ с целью определить, что они соответствуют описанным требованиям, показать, что они подходят для заявленных целей и для определения дефектов.

*** ISTQB** — International Software Testing Qualifications Board Glossary

ПРИЧИНЫ ОШИБОК В ПО

- ✓ Человеческий фактор
- ✓ Проблемы в описании требований к программному обеспечению
- ✓ Недостаток времени
- ✓ Недостаточно продуманная архитектура приложения
- ✓ Недостаточное знание бизнеса
- ✓ Нехватка профессиональных навыков и опыта
- ✓ Изменения «в последнюю минуту»

ИСТОРИЯ ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1950-е

Первые программные системы разрабатывались в рамках программ научных исследований или программ для нужд министерств обороны.

Тестирование таких продуктов проводилось **строго формализовано** с записью всех тестовых процедур, тестовых данных, полученных результатов.

Тестирование выделялось в отдельный процесс, который начинался после завершения кодирования, но при этом, как правило, выполнялось тем же персоналом.

Фактически тестирование **представляло собой** скорее **отладку программ** (*debugging*).

ИСТОРИЯ ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1960-е

В 1960-х много внимания уделялось **<u>«исчерпывающему» тестированию</u>**, которое должно проводиться с использованием всех путей в коде или всех возможных входных данных.

Однако это невозможно:

- количество возможных входных данных очень велико;
- существует множество путей;
- сложно найти проблемы в архитектуре и спецификациях.

Итог: «исчерпывающее» тестирование было отклонено и признано теоретически невозможным.

ИСТОРИЯ ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1970-е

В 1970-х годах фактически родились **две** фундаментальные идеи тестирования:

- ✓ тестирование сначала рассматривалось как **процесс доказательства работоспособности** программы в некоторых заданных условиях (*positive testing*), а затем:
- ✓ **как процесс доказательства неработоспособности** программы в некоторых заданных условиях (*negative testing*)

Бытует неверное понимание того, что негативные тест-кейсы должны заканчиваться возникновением сбоев и отказов в приложении. Нет, это не так. Ожидаемым результатом негативных тест-кейсов является именно корректное поведение приложения, а сами **негативные тест-кейсы считаются пройденными успешно, если им не удалось «поломать» приложение.**

ИСТОРИЯ ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1980-е

В 80-х годах произошло ключевое изменение места тестирования в разработке ПО:

- ✓ вместо одной из финальных стадий создания проекта **тестирование стало применяться на протяжении всего цикла разработки**, что позволило не только быстро обнаруживать и устранять проблемы, но даже предсказывать и предотвращать их появление.

В этот же период времени отмечено бурное **развитие и формализация методологий тестирования** и появление первых элементарных попыток автоматизировать тестирование.

В ходе тестирования надо проверить не только собранную программу, но и требования, код, архитектуру, сами тесты.

ИСТОРИЯ ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1990-е

В 90-х годах произошёл переход от тестирования как такового к более всеобъемлющему процессу, который называется **«обеспечение качества (quality assurance8)»**. В понятие «тестирование» стали включать планирование, проектирование, создание, поддержку и выполнение тест-кейсов и тестовых окружений.

Начинают появляться различные программные инструменты для поддержки процесса тестирования: более продвинутые среды для автоматизации с возможностью создания скриптов и генерации отчетов, системы управления тестами, ПО для проведения нагрузочного тестирования.

ИСТОРИЯ ТЕСТИРОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2000-е

С нулевых годов появляются гибкие методологии разработки и такие подходы, как «разработка под управлением тестированием (*test-driven development, TDD*)».

У тестировщиков появляются специализированные технологии и инструменты, а сам **процесс тестирования интегрирован в цикл разработки программного обеспечения**.

Популярны идеи о том, что **во главу процесса тестирования следует ставить** не соответствие программы требованиям, а **способность ПО предоставить конечному пользователю возможность эффективно решать свои задачи**.

Кто занимается тестированием

- ✓ Заказчики (приёмочное тестирование)
- ✓ Разработчики (модульное или unit-тестирование)
- ✓ Тестировщики (ручное и автоматизированное тестирование)
- ✓ Бизнес аналитики (тестирование требований)
- ✓ Команда технической поддержки продукта (регрессионное тестирование)
- ✓ Дизайнеры (тестирование пользовательского интерфейса)
- ✓ Конечные пользователи (beta-тестирование)

QA QC Testing

ТЕСТИРОВАНИЕ

КОНТРОЛЬ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА

Обеспечение качества (Quality Assurance - QA) - это совокупность мероприятий, охватывающих все технологические этапы разработки, выпуска и эксплуатации программного обеспечения информационных систем, предпринимаемых на разных стадиях жизненного цикла ПО, для обеспечения требуемого уровня качества выпускаемого продукта.

Контроль качества (Quality Control - QC) - это совокупность действий, проводимых над продуктом в процессе разработки, для получения информации о его актуальном состоянии в разрезах: "готовность продукта к выпуску", "соответствие зафиксированным требованиям", "соответствие заявленному уровню качества продукта".

Тестирование программного обеспечения (Software Testing) - это одна из техник контроля качества, включающая в себя активности по планированию работ (Test Management), проектированию тестов (Test Design), выполнению тестирования (Test Execution) и анализу полученных результатов (Test Analysis).

В чем отличия QA от тестирования

- **QA** — это забота о качестве в виде превентивных мероприятий, направленных на то, что бы дефекты не появились.
- **Тестирование** — это забота о качестве в виде обнаружения дефектов до того, как их найдут пользователи.

Общее в QA и тестировании заключается в том, что **<u>они призваны улучшить ПО</u>**, различие между ними — в том, что:

- QA призвано улучшить ПО через улучшение процесса разработки ПО;
- тестирование — через обнаружение дефектов.

Место в процессе

- Активности **QA** проходят на всем протяжении SDLC: на этапе построения, анализа и улучшения процессов, формирования релизных политик, риск менеджмента и прочих how-to’s.

- **QC** подключаются на этапе составления критериев качества, метрик и способов оценки.

- **Тестировщик** вступает уже после этапа разработки (с **shift left** на этапе получения ТЗ и превращения его в спецификацию).

Кто такой тестировщик

В ЕКСД* РБ выделены 2 должности :

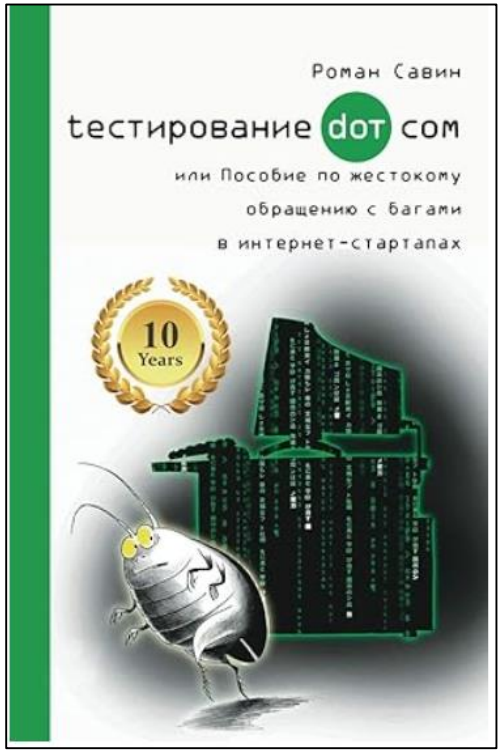
- ✓ специалист по тестированию программного обеспечения (**QA- инженер)**;
- ✓ тестировщик программного обеспечения (**QA- тестировщик)**.

QA-Тестировщик — это специалист, который находит ошибки (баги) в работе программного обеспечения (ПО) во время его тестирования, чтобы повысить качество продукта. В его обязанности входит: 1. Проверяют соответствие ПО заданным требованиям 2. Фиксируют ошибки 3. Создают тестовые планы.	**QA-инженер** — это специалист, который тестирует и контролирует качество продукта на всех этапах его создания, который дает рекомендации по улучшению продукта и повышению качества разработки. Обязанности: 1. Изучают продукт 2. Вносят замечания на начальных этапах тестирования 3. Определяют, какие нужно провести тесты 4. Ставят сроки начала тестирования 5. Координируют взаимодействие тестировщиков и разработчиков 6. Следят, чтобы обо всех ошибках, которые обнаружились на разных этапах разработки и тестирования, узнали нужные люди 7. Контролируют сроки решения проблем 8. Ставят приоритеты в работе.

** ЕКСД – единый классификационный справочник должностей*

*Результатом работы тестировщика является **счастье** (удовлетворение) **конечного пользователя**. Причем "счастье" не в глобальном его значении, а та его часть, которая связана с качеством вашего продукта.*

Роман Савин
Книга: тестирование dot com или Пособие по жестокому обращению с багами в интернет-стартапах



КАЧЕСТВО СИСТЕМЫ

Это степень удовлетворения системой заявленных и подразумеваемых потребностей различных заинтересованных сторон, которая позволяет, таким образом, оценить достоинства.

Эти заявленные и подразумеваемые потребности представлены в международных стандартах серии SQuaRE посредством моделей качества, которые представляют качество продукта в виде разбивки на классы характеристик.

Стандарт качества ПО ISO/IEC 25010:2011

НАЦИОНАЛЬНЫЙ СТАНДАРТ РОССИЙСКОЙ ФЕДЕРАЦИИ

ГОСТ Р ИСО/МЭК 25010— 2015

Информационные технологии

СИСТЕМНАЯ И ПРОГРАММНАЯ ИНЖЕНЕРИЯ

Требования и оценка качества систем и программного обеспечения (SQuaRE). Модели качества систем и программных продуктов

ISO/IEC 25010:2011
Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models
(IDT)

Содержание

1 Область применения ....
2 Соответствие ....
3 Основы модели качества ....
3.1 Модели качества ....
3.2 Модель качества при использовании ....
3.3 Модель качества продукта ....
3.4 Цели моделей качества ....
3.5 Применение модели качества ....
3.6 Качество с точки зрения различных заинтересованных
3.7 Взаимосвязь моделей ....
4 Термины и определения ....
4.1 Термины к модели качества при использовании ....
4.2 Термины к модели качества продукта ....
4.3 Общие определения ....
4.4 Термины и определения из ИСО/МЭК 25000 ....
Приложение А (справочное) Сравнение с моделью качества
Приложение В (справочное) Пример отображения функциона
Приложение С (справочное) Использование модели качества
Библиография ....

МОДЕЛИ КАЧЕСТВА

К настоящему времени в серии SQuaRE имеются три модели качества:

- ✓ Модель качества при использовании
- ✓ Модель качества продукта
- ✓ Модель качества данных, определенная в ИСО/МЭК 25012

Совместное использование моделей качества дает основание считать, что учтены все характеристики качества.

МОДЕЛЬ КАЧЕСТВА ПРИ ИСПОЛЬЗОВАНИИ

- ✓ Эффективность (результативность)
- ✓ Производительность
- ✓ Удовлетворенность (полноценность, доверие, удовольствие, комфорт)
- ✓ Свобода от риска (смягчение отрицательных последствий):
  - экономического риска;
  - риска для здоровья и безопасности;
  - экологического риска.
- ✓ Покрытие контекста (полнота контекста, гибкость)

Определение характеристик моделей качества подробно дано в ГОСТ Р ИСО /МЭК 25010—2015

Например:

4.1.1 **эффективность, результативность** (effectiveness): Точность и полнота, с которой пользователи достигают определенных целей (ИСО 9241-11).

4.1.2 **эффективность, производительность** (efficiency): Связь точности и полноты достижения пользователями целей с израсходованными ресурсами (ИСО 9241-11).

Примечание - Соответствующие ресурсы могут включать в себя время выполнения задачи (человеческие ресурсы), материалы или финансовые затраты на использование.

4.1.3 **удовлетворенность** (satisfaction): Способность продукта или системы удовлетворить требованиям пользователя в заданном контексте использования.

4.1.3.1 **полноценность** (usefulness): Степень удовлетворенности пользователя достижением прагматических целей, включая результаты использования и последствия использования.

4.1.3.2 **доверие** (trust): Степень уверенности пользователя или другого заинтересованного лица в том, что продукт или система будут выполнять свои функции так, как это предполагалось.

4.1.3.3 **удовольствие** (pleasure): Степень удовольствия пользователя от удовлетворения персональных требований.

4.1.3.4 **комфорт** (comfort): Степень удовлетворенности пользователя физическим комфортом.

И т.д. …….

4.1.4 **свобода от риска** (freedom from risk): Способность продукта или системы смягчать потенциальный риск для экономического положения, жизни, здоровья или окружающей среды.

Примечание - Риск является функцией вероятности возникновения такой угрозы и потенциальных неблагоприятных последствий этой угрозы.

4.1.4.1 **смягчение отрицательных последствий экономического риска** (economic risk mitigation): Способность продукта или системы смягчать потенциальный риск для финансового положения и эффективной работы, коммерческой недвижимости, репутации или других ресурсов в предполагаемых условиях использования.

4.1.4.2 **смягчение отрицательных последствий риска для здоровья и безопасности**(health and safety risk mitigation): Способность продукта или системы смягчать потенциальный риск для людей в предполагаемых условиях использования.

4.1.4.3 **смягчение отрицательных последствий экологического риска** (environmental risk mitigation): Способность продукта или системы смягчать потенциальный риск для имущества или окружающей среды в предполагаемых условиях использования.

МОДЕЛЬ КАЧЕСТВА ПРОДУКТА

- ✓ Функциональная пригодность
- ✓ Уровень производительности
- ✓ Совместимость
- ✓ Удобство использования
- ✓ Надежность
- ✓ Защищенность
- ✓ Сопровождаемость
- ✓ Переносимость

МОДЕЛЬ КАЧЕСТВА ПРОДУКТА

- ✓ Функциональная пригодность:
  - функциональная полнота;
  - функциональная корректность;
  - функциональная целесообразность.
- ✓ Уровень производительности:
  - временные характеристики;
  - использование ресурсов;
  - потенциальные возможности.
- ✓ Совместимость:
  - сосуществование;
  - интероперабельность.

МОДЕЛЬ КАЧЕСТВА ПРОДУКТА

- ✓ Удобство использования:
  - определимость пригодности;
  - изучаемость;
  - управляемость;
  - защищенность от ошибки пользователя;
  - эстетика пользовательского интерфейса
  - доступность.
- ✓ Надежность:
  - завершенность;
  - готовность;
  - отказоустойчивость;
  - восстанавливаемость.

МОДЕЛЬ КАЧЕСТВА ПРОДУКТА

✓ Защищенность:

- конфиденциальность;
- целостность;
- неподдельность;
- отслеживаемость;
- подлинность.

✓ Переносимость:

- адаптируемость;
- устанавливаемость;
- взаимозаменяемость.

✓ Сопровождаемость:

- модульность;
- возможность многократного использования;
- анализируемость;
- модифицируемость;
- тестируемость.

Показатели надёжности ПО

На практике различают несколько типов показателей надёжности ПО:

- количество ошибок перед отладкой и после неё;
- наработка часов на отказ;
- интенсивность отказов;
- вероятность безотказного действия в течение заданного отрезка времени.

Показатели оцениваются как количественные, качественные и порядковые. Наиболее ценную информацию дают количественные. Их получают путем непосредственных наблюдений и обработки результатов испытаний систем.

7 принципов тестирования

Тестирование показывает наличие дефектов, а не их отсутствие

Очень сложно обнаружить нечто, относительно чего мы не знаем — ни «где оно», ни «как оно выглядит», ни даже «существует ли оно вообще». Так как не существует физической возможности проверить поведение сложного программного продукта во всех возможных ситуациях и условиях, тестирование не может гарантировать, что в той или иной ситуации, при стечении тех или иных обстоятельств дефект не возникнет. Тестирование может проверить наиболее вероятные, востребованные ситуации и обнаружить дефекты при их возникновении. Такие дефекты будут устранены, что ощутимо повысит качество продукта, но по-прежнему не гарантирует от возникновения проблем в оставшихся, не проверенных ситуациях и условиях

Исчерпывающее тестирование невозможно

Даже для одного простого поля для ввода имени пользователя может существовать порядка 2.4^{32} позитивных проверок и бесконечное количество негативных проверок. Потому невозможно протестировать программный продукт полностью, «исчерпывающе».
Однако, из этого не следует, что тестирование как таковое не является эффективным. Вдумчивый анализ требований, учёт рисков, расстановка приоритетов, анализ предметной области, моделирование, работа с конечными пользователями, применение специальных техник тестирования — эти и многие другие подходы позволяют выявить те области или условия эксплуатации продукта, которые требуют особенно тщательной проверки.

Принципы тестирования

Тестирование тем эффективнее, чем раньше оно выполняется

Раннее тестирование помогает устранить или сократить дорогостоящие изменения. У данного принципа есть прекрасная аналогия из обычной повседневной жизни. Представьте, что вы собираетесь в поездку и продумываете список вещей, которые необходимо взять с собой. На стадии обдумывания добавить, изменить, удалить любой пункт в этом списке не стоит ничего. На стадии поездки по магазинам для закупки необходимого недоработки в списке уже могут привести к необходимости повторной поездки в магазин. На стадии отправки на место назначения недоработки в списке вещей явно приведут к ощутимой потере нервов, времени и денег. А если фатальный недостаток списка вещей выяснится только по прибытии, может так оказаться, что вся поездка потеряла смысл.

Кластеризация дефектов

Дефекты как правило группируются в какой-то «проблемной» области приложения.
Группировка дефектов по какому-то явному признаку является хорошим поводом к продолжению исследования данной области программного продукта: скорее всего, именно здесь будет обнаружено ещё больше дефектов. Обнаружение подобных тенденций к кластеризации (и особенно поиск глобальной первопричины) часто требует от тестировщиков определённых знаний и опыта, но если такой «кластер» выявлен — это позволяет ощутимо минимизировать усилия и при этом существенно повысить качество приложения.

Принципы тестирования

Парадокс пестицида

Название данного принципа происходит от общеизвестного явления в сельском хозяйстве: если долго распылять один и тот же пестицид на посевы, у насекомых вскоре вырабатывается иммунитет, что делает пестицид неэффективным.
Проявляется в повторении одних и тех же (или просто однотипных) проверок снова и снова: со временем эти проверки перестанут обнаруживать новые дефекты. Чтобы преодолеть парадокс пестицида, необходимо регулярно пересматривать и обновлять тест-кейсы, разнообразить подходы к тестированию, применять различные техники тестирования, смотреть на ситуацию «свежим взглядом».

Тестирование зависит от контекста

Набор характеристик программного продукта влияет на глубину тестирования, используемый набор техник и инструментов, принципы организации работы тестировщиков и т.д.

Отсутствие дефектов — не самоцель

Программный продукт должен не только быть избавлен от дефектов настолько, насколько это возможно, но и удовлетворять требованиям заказчика и конечных пользователей.
Именно понимание контекста продукта и потребностей пользователей позволяет тестировщикам выбрать наилучшую стратегию и добиться наилучшего результата.

Зачем будущим разработчикам изучать тестирование?

Изучение тестирования разработчиками — не «дополнительная нагрузка», а **стратегическая необходимость** в современной инженерной практике.

Аргумент 1.

Экономика качества:

Чем раньше найден дефект — тем дешевле его исправить

Этап обнаружения дефекта	**Относительная стоимость исправления***
Требования	1×
Проектирование	3–6×
Разработка (до коммита)	10–15×
Системное тестирование	30–40×
Продакшен (после релиза)	100–1000×

**Источник: IBM Systems Sciences Institute, NIST*

<u>Аргумент 2</u>. Трансформация роли разработчика в современных командах

Современный разработчик — не «писатель кода», а владелец качества функции от идеи до эксплуатации:

Роль разработчика в 2000-х	Роль разработчика в 2020-х (и далее)
Пишет код → передаёт QA	Отвечает за покрытие функции тестами
«Это не мой баг»	Анализирует дефекты в своём коде и предотвращает повторение
Работает в изоляции	Коллаборирует с QA на этапе требований («три амиго»: разработчик + тестировщик + аналитик)

Факт рынка: 89% вакансий для middle/senior разработчиков в 2025 г. требуют навыков написания unit/integration-тестов (*исследование Stack Overflow Developer Survey 2025*). Отсутствие этих компетенций снижает рыночную стоимость специалиста на 20–30%.

Аргумент 3. Тестирование как инструмент проектирования, а не проверки

Тесты — не «проверка работоспособности», а средство прояснения требований и архитектуры:

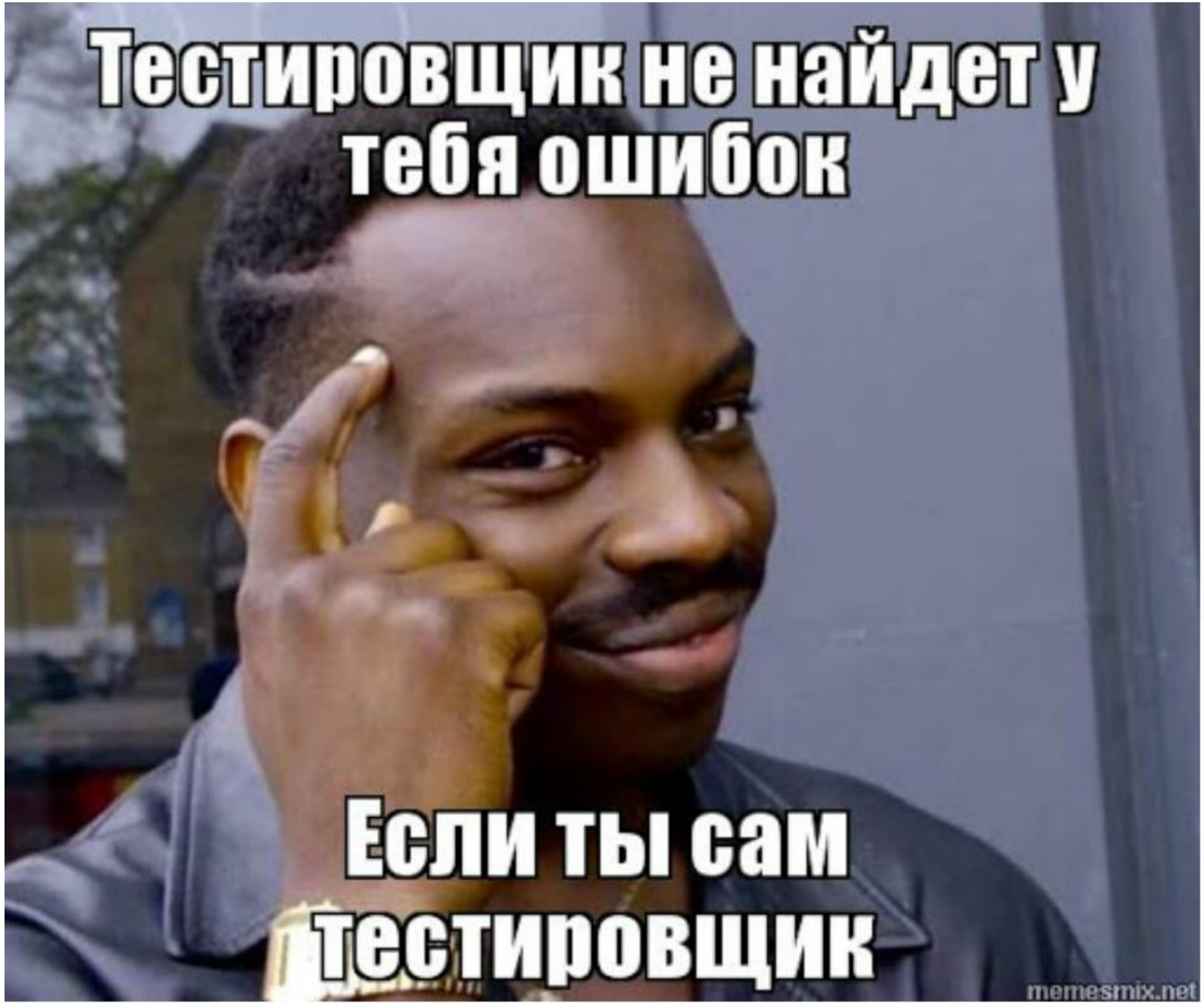
- **Приёмочные критерии в формате Given/When/Then** (BDD) заставляют разработчика задуматься о граничных сценариях *до* реализации.
- **Unit-тесты** выявляют проблемы связанности (жесткая связность): если код невозможно протестировать изолированно — архитектура некорректна.
- **Тестирование на основе состояний** помогает моделировать сложную бизнес-логику (например, жизненный цикл заказа) без ошибок переходов.

Аргумент 4. Ускорение личной продуктивности через автоматизацию тестирования

Разработчик без навыков тестирования тратит до 40% времени на ручную проверку изменений после каждого рефакторинга

<u>Аргумент 5</u>. Формирование инженерного мышления «через призму пользователя»

Тип мышления разработчика	Как формируется через тестирование
Позитивное («как должно работать»)	Тестирование при программировании
Негативное («как может сломаться»)	Через граничные значения, негативные сценарии
Эвристическое («где вероятнее всего ошибка»)	Через анализ историй дефектов и риск-ориентированный подход
Эмпатическое («как пользователь столкнётся с багом»)	Через исследовательское тестирование и анализ юзабилити


Тестировщик не найдет у тебя ошибок
Если ты сам тестировщик
memesmix.net